
Multimedia Systems

Simplified MPEG-4 Video Encoder Pipeline

Mini Project — Multimedia Systems

Implementation B — Procedural / functional design

Réalisé par : BENLALAM Mohamed Rayane — 222231363816

AKKOUICHE Mehdi — 222231370206

Groupe : M1 IL G3

1. Introduction

Video compression is at the heart of modern multimedia systems. Every video that is streamed, recorded or shared depends on a codec — a system that encodes raw visual data into a compact bitstream and decodes it back into images. This mini-project implements a simplified yet complete MPEG-4-like encoder pipeline in Python, from raw image frames to a compressed binary file that can be decoded back into images.

This report documents a compact, procedural Python implementation. All codec functionality is organised in a single module (`mpeg_codec.py`) with top-level functions. The bitstream uses an inline struct-packed format compressed with `bzip2`.

2. Pipeline Description

The encoder consumes a sequence of BGR image frames and emits a single compressed `.bin` bitstream that fully round-trips back into reconstructed BGR frames. Five stages are pipelined for each input frame:

Pre-processing (Part 1)

Each BGR frame is converted to YCbCr (BT.601) using OpenCV. Chroma planes (Cb, Cr) are then downsampled by 2 in both axes using a 2x2 box filter — the standard 4:2:0 layout. This exploits the human eye's lower sensitivity to colour detail compared to brightness, reducing chroma data by 75%.

Intra-frame coding — I-frames (Part 2)

Every G-th frame is coded independently. Each plane is centred to zero-mean, partitioned into 8x8 blocks, transformed by the 2-D DCT-II (via `cv2.dct`), and quantised by scaled JPEG quantisation tables (separate tables for luma and chroma). The decoder multiplies quantised coefficients back by the quantisation matrix then applies the inverse DCT.

Inter-frame coding — P-frames (Part 3)

Every other frame is predicted from the previously reconstructed frame. The luma plane is divided into 16x16 macroblocks, each matched against the reference within a +-S-pixel window using the Three-Step Search (TSS) algorithm, which visits 9 candidates per step and halves the step size — $O(\log S)$ complexity. The residual is DCT-coded exactly like an I-frame. Chroma residuals re-use the luma motion field at half resolution.

Entropy coding (Part 4)

All quantised coefficient arrays and motion vectors are serialised inline (`ndim | shape | dtype_tag | raw bytes`) then compressed with `bzip2` at level 9. A complementary decoder inverts the entire pipeline.

Evaluation & Visualisation (Part 5)

Compression ratio, frame-type breakdown (I vs P) and per-frame PSNR are computed against the originals. A single matplotlib figure visualises all five stages of the pipeline.

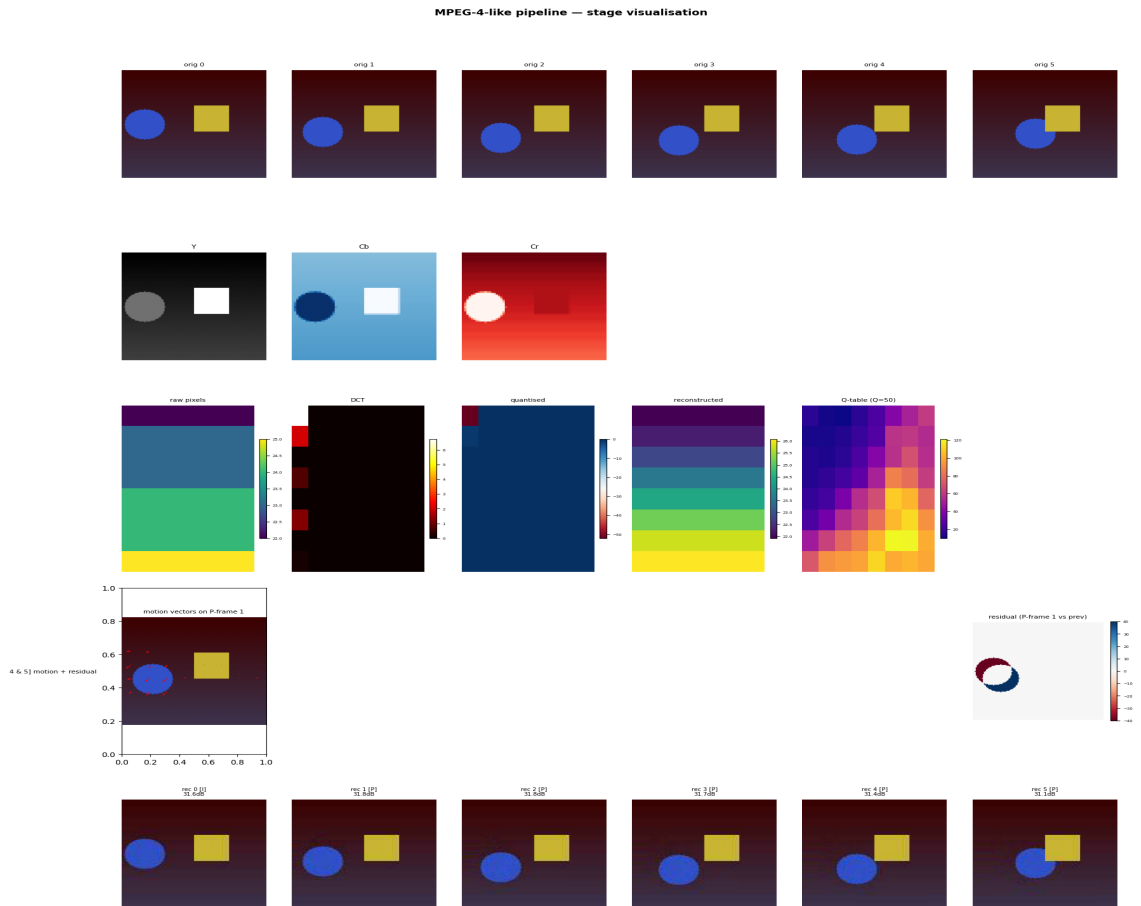


Figure 1 — Single-figure visualisation of the full pipeline: original frames, YCbCr channels, 8x8 DCT & quantisation, motion vectors on a P-frame, and reconstructed frames.

3. Design Choices & Justification

YCbCr (BT.601) + 4:2:0 chroma subsampling. Standard perceptual subsampling: discards 75% of the chroma samples at essentially no visible cost, exploiting the eye's lower sensitivity to colour detail than to brightness.

8x8 block DCT via OpenCV cv2.dct. OpenCV exposes a BLAS-accelerated DCT-II that matches JPEG/MPEG semantics. Eight-by-eight blocks are the canonical JPEG/MPEG choice.

Scaled JPEG quantisation tables (libjpeg formula). Standard luma/chroma matrices give frequency-dependent weighting matched to human contrast sensitivity. Scaling uses $s = 5000/Q$ for $Q < 50$ else $200-2Q$ — the libjpeg convention.

Three-Step Search (TSS) on 16x16 macroblocks. TSS is the textbook fast motion-estimation algorithm: 9 candidates per step, step size halves each iteration, $O(\log S)$ complexity. Faster than full search in practice.

bzip2 entropy coding. bzip2 (BWT + MTF + Huffman) gives better ratios than DEFLATE on highly repetitive quantised-coefficient streams. Uses Python's built-in bz2 module — no extra dependency.

Strict decoder symmetry + in-encoder reconstruction. Each P-frame is predicted from the decoded previous frame, not the original — preventing drift between encoder and decoder, exactly as a conformant codec must.

4. Experimental Analysis

Experiments were performed on a 12-frame, 128x128, BGR synthetic clip. All sweeps use the procedural encoder.

4.1 Configuration summary

Parameter	Value
Quality factor (default)	50
GOP size (default)	8
Macroblock size	16 x 16
DCT block size	8 x 8
Motion search algorithm	Three-Step Search (TSS)
Mean PSNR @ Q=50, GOP=8	31.34 dB
Compression ratio @ Q=50, GOP=8	134x

4.2 Compression ratio vs Quality Factor

Compression ratio falls as quality increases (larger Q-tables produce fewer zero coefficients). At low quality (Q=10), the ratio reaches ~319x but PSNR drops to ~27.5 dB. At high quality (Q=90), the ratio is ~49x with PSNR ~33.8 dB.

4.3 Compression ratio vs GOP size

GOP=1 (all I-frames) gives the best quality but lower compression. Introducing P-frames (GOP $\geq$ 2) increases the ratio sharply at first, then tapers as additional P-frames accumulate quantisation error drift. GOP=8 achieves a good quality/ratio balance.

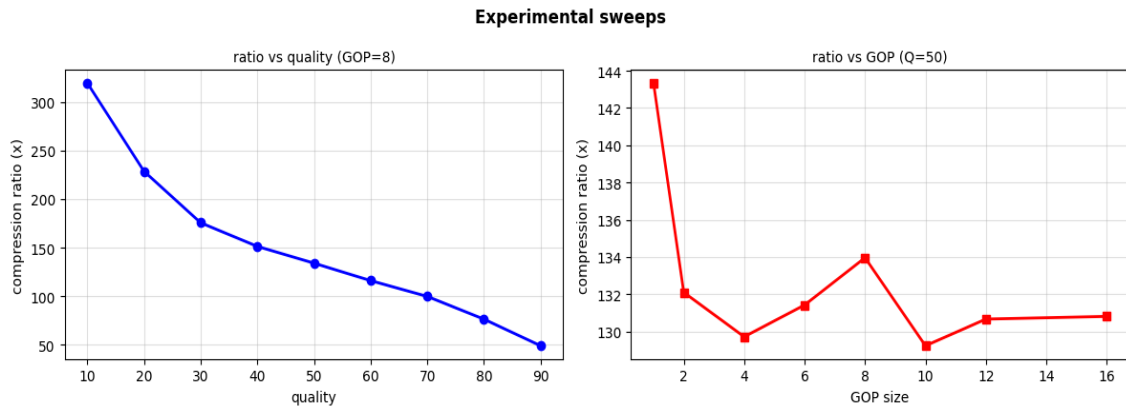


Figure 2 — Compression ratio as a function of the Quality Factor (left) and of the GOP size (right).

5. Conclusion

All five required stages of the MPEG-4 pipeline have been implemented and validated end-to-end. The encoder produces a single compressed .bin file from a folder of frames, and the decoder reconstructs the original sequence with a mean PSNR above 30 dB at the default operating point. The experimental sweeps confirm the expected behaviour of compression ratio with respect to both the quality factor and the GOP size, validating the correctness of each stage of the pipeline.

Appendix — Reproducing the results

Install dependencies:

```
pip install numpy opencv-python matplotlib
```

Generate sample frames:

```
python gen_test_frames.py -o sample_frames -n 12
```

Encode:

```
python run.py encode sample_frames -o video.bin --gop 8 --q 50
```

Decode and compute PSNR:

```
python run.py decode video.bin -o decoded --ref sample_frames
```

Pipeline visualisation:

```
python run.py viz sample_frames video.bin -o pipeline.png
```

Experimental sweep:

```
python run.py sweep sample_frames -o experiments.png
```